

Multimodal Sentiment Analysis Code:

```
import os
import shutil
import zipfile
import random
import matplotlib.pyplot as plt
import numpy as np
from PIL import Image
from textblob import TextBlob
import tensorflow as tf
from tensorflow.keras.models import Sequential, load_model
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint

# Step 1: Unzip dataset if not extracted
zip_path = 'fer_dataset.zip'
extract_path = 'fer_dataset'

if os.path.exists(extract_path):
    shutil.rmtree(extract_path)
with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    zip_ref.extractall(extract_path)

print("✅ Extracted 'fer_dataset.zip'.")

# Step 2: Organize dataset into sentiment folders
sentiment_map = {
    'happy': 'Positive',
    'sad': 'Negative',
    'angry': 'Negative',
    'disgusted': 'Negative',
    'fearful': 'Negative',
    'surprised': 'Surprised'
}

dataset_dir = 'dataset'
if os.path.exists(dataset_dir):
    shutil.rmtree(dataset_dir)
os.makedirs(dataset_dir, exist_ok=True)

for sentiment in set(sentiment_map.values()):
    os.makedirs(os.path.join(dataset_dir, sentiment), exist_ok=True)

test_folder = os.path.join(extract_path, 'test')
for emotion, sentiment in sentiment_map.items():
    emotion_folder = os.path.join(test_folder, emotion)
    if os.path.exists(emotion_folder):
        for file in os.listdir(emotion_folder):
            shutil.copy(os.path.join(emotion_folder, file), os.path.join(dataset_dir, sentiment,
```

```

file))
    else:
        print(f" ⚠ Warning: '{emotion_folder}' not found!")

print(f" ✅ Images organized into Positive, Negative, and Surprised.")

# Step 3: Data Preprocessing
datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=15,
    width_shift_range=0.1,
    height_shift_range=0.1,
    zoom_range=0.2,
    horizontal_flip=True,
    validation_split=0.2
)

train_data = datagen.flow_from_directory(
    dataset_dir,
    target_size=(64, 64),
    batch_size=32,
    class_mode='categorical',
    subset='training'
)

val_data = datagen.flow_from_directory(
    dataset_dir,
    target_size=(64, 64),
    batch_size=32,
    class_mode='categorical',
    subset='validation'
)

# Step 4: CNN Model
model = Sequential([
    Conv2D(32, (3,3), activation='relu', input_shape=(64, 64, 3)),
    MaxPooling2D(2,2),
    Conv2D(64, (3,3), activation='relu'),
    MaxPooling2D(2,2),
    Conv2D(128, (3,3), activation='relu'),
    MaxPooling2D(2,2),
    Flatten(),
    Dense(256, activation='relu'),
    Dropout(0.4),
    Dense(3, activation='softmax')
])

model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

# Step 5: Callbacks
callbacks = [

```

```

    EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True),
    ModelCheckpoint('fer_sentiment_model.h5', save_best_only=True)
]

```

Step 6: Train the model

```

model.fit(train_data, epochs=30, validation_data=val_data, callbacks=callbacks)
print("✅ Model trained and saved.")

```

Step 7: Predict Image Sentiment

```

def predict_image_sentiment(image_path):
    model = load_model("fer_sentiment_model.h5")
    img = Image.open(image_path).convert('RGB').resize((64, 64))
    img_array = np.array(img) / 255.0
    img_array = np.expand_dims(img_array, axis=0)

    prediction = model.predict(img_array)[0]
    classes = ['Negative', 'Positive', 'Surprised']
    predicted_class = classes[np.argmax(prediction)]
    confidence = np.max(prediction)
    return predicted_class, confidence

```

Step 8: Text Sentiment

```

def predict_text_sentiment(text):
    analysis = TextBlob(text)
    polarity = analysis.sentiment.polarity
    if polarity > 0.1:
        sentiment = 'Positive 😊'
    elif polarity < -0.1:
        sentiment = 'Negative 😞'
    else:
        sentiment = 'Surprised 😲'
    print(f"📝 Text Sentiment: {sentiment} (Polarity: {polarity:.2f})")

```

Step 9: Predict Random Images

```

def predict_random_images():
    model = load_model("fer_sentiment_model.h5")
    classes = ['Positive', 'Negative', 'Surprised']

    for cls in classes:
        folder_path = os.path.join(dataset_dir, cls)
        if not os.path.exists(folder_path): continue

        image_list = os.listdir(folder_path)
        if not image_list:
            print(f"⚠️ No images in '{cls}' folder.")
            continue

        images = random.sample(image_list, min(5, len(image_list)))
        print(f"\n🖼️ Predictions on 5 random images from '{cls}':")

        for img_name in images:

```

```
img_path = os.path.join(folder_path, img_name)
sentiment, confidence = predict_image_sentiment(img_path)
print(f"Image: {img_name} --> {sentiment} (Confidence: {confidence:.2f})")
```

```
img = Image.open(img_path)
plt.imshow(img)
plt.axis('off')
plt.title(f"{sentiment}\nConfidence: {confidence:.2f}")
plt.show()
```

Step 10: Run Final Test

```
predict_text_sentiment("I feel very surprised but also a bit happy.")
predict_random_images()
```